

■ Python Concurrency — Lecture 3

Threading Deep Dive

Threads | Args | I/O | Lock | Race Condition

#	Section	Description
1	Thread vs Process	Architecture diagram — memory & structure
2	Thread API Basics	Thread(), .start(), .join() explained
3	Example 1 — Breakfast	Basic two-thread program (05_thread1.py)
4	Example 2 — Chai Args	Passing arguments via args tuple (06_thread2.py)
5	Example 3 — Downloads	Threading with I/O (07_thread_download.py)
6	Where Threading Fails	CPU-bound tasks — GIL bottleneck
7	Thread Lock	Race condition & threading.Lock() (08_thread_lock.py)
8	Interview Q&A;	Top questions with model answers
9	Quick Revision	Cheatsheet & golden rules

■ Section 1 — Thread vs Process Architecture

Process — The Big Picture

A **Process** is an independent instance of a running program. Each process gets:

- Its own **dedicated memory space** — completely isolated from other processes
- Its own **stack, heap, and code segment**
- Its own **Python interpreter and GIL** (in multiprocessing)

Processes cannot directly access each other's memory. They must use IPC mechanisms like Queue or Pipe to communicate.

Thread — The Lightweight Worker

A **Thread** is a unit of execution that lives *inside* a process. Multiple threads in the same process:

- **Share the same memory space** — can read each other's variables
- **Share code and data segments**
- Have their own **individual stack and registers**

Because memory is shared, threads are lightweight and fast to create — but shared memory also means race conditions are possible without proper locking.

Feature	Thread	Process
Lives inside	A process	Independent
Memory	Shared with all threads	Own private memory
Creation speed	Fast (lightweight)	Slower (heavier)
Communication	Direct (shared vars)	IPC — Queue, Pipe
GIL	Shared — one GIL per process	Own GIL per process
Crash isolation	Crash affects whole process	Isolated — crash = only that process
Use case	I/O-bound concurrent tasks	CPU-bound parallel tasks
Python module	threading	multiprocessing

■ Section 2 — Thread API Basics

Core Threading Methods

Method / Property	Purpose	Notes
<code>threading.Thread(target=f)</code>	Create a thread object	Does NOT start execution
<code>threading.Thread(args=(a,))</code>	Pass arguments to target function	Must be a tuple — trailing comma for one arg
<code>threading.Thread(name='X')</code>	Give the thread a label	Optional but useful for debugging
<code>.start()</code>	Begin thread execution	Returns immediately — thread runs in background
<code>.join()</code>	Wait until thread terminates	Blocks caller until this thread finishes
<code>.name</code>	Thread's name attribute	Set via <code>name=</code> or auto-assigned
<code>threading.current_thread()</code>	Returns the currently running thread	Useful inside target functions for logging
<code>threading.Lock()</code>	Create a mutual exclusion lock	Used to protect shared data
<code>with lock:</code>	Acquire lock, execute, auto-release	Recommended over manual acquire/release

■ Section 3 — Example 1: Basic Threading (05_thread1.py)

[illegible]

We need to prepare breakfast. Two independent tasks:

- **Boil milk** — takes 2 seconds
- **Toast a bun** — takes 3 seconds

Sequential approach: $2 + 3 = \mathbf{5 \text{ seconds}}$

With threading: both run concurrently = **3 seconds** (time of the slowest task)

- We need to prepare breakfast. Two independent tasks:
- **Boil milk** — takes 2 seconds
 - **Toast a bun** — takes 3 seconds
- Sequential approach: $2 + 3 = \mathbf{5 \text{ seconds}}$
- With threading: both run concurrently = **3 seconds** (time of the slowest task)

We need to prepare breakfast. Two independent tasks:

- **Boil milk** — takes 2 seconds
- **Toast a bun** — takes 3 seconds

Sequential approach: $2 + 3 = \mathbf{5 \text{ seconds}}$

With threading: both run concurrently = **3 seconds** (time of the slowest task)

We need to prepare breakfast. Two independent tasks:

- **Boil milk** — takes 2 seconds
- **Toast a bun** — takes 3 seconds

Sequential approach: $2 + 3 = \mathbf{5 \text{ seconds}}$

With threading: both run concurrently = **3 seconds** (time of the slowest task)

[Full Code — 05_thread1.py](#)

[illegible]

Expected Output & Analysis

```
Boiling milk...
Toasting bun...      <- Both start almost simultaneously
Milk boiled!         <- 2 seconds later
Done with bun toast!  <- 3 seconds later
Breakfast ready in 3.01 seconds!

Sequential time: 5 seconds
Threading time: 3 seconds  <- 40% faster!
```

Key concepts demonstrated:

- **threading.Thread(target=func)** — creates a thread object that KNOWS its job but has not started yet. The thread is idle until `start()` is called.
- **.start()** — tells the operating system to schedule this thread for execution. It returns immediately — the thread runs concurrently in the background.
- **.join()** — the main thread pauses here and waits for this specific thread to finish. Without `join()`, the main program might exit before threads complete.

■ Section 4 — Example 2: Passing Arguments (06_thread2.py)

Scenario

We have a single function `prepare_chai(type_, wait_time)` that can brew any type of chai. We want to use the same function for different chai types by passing arguments — demonstrating how to pass parameters to threaded functions.

[Full Code — 06_thread2.py](#)

[illegible]

The args Tuple — Critical Rule

args must ALWAYS be a tuple — this is one of the most common mistakes in interviews.

Python uses the tuple to unpack arguments into the function's parameters positionally.

```
# WRONG – not a tuple, just parentheses:
args=('Masala')      # Python sees this as just a string
args=(2)             # Python sees this as just an integer

# CORRECT – single item tuple needs trailing comma:
args=('Masala',)     # Trailing comma makes it a tuple
args=(2,)            # Trailing comma required

# CORRECT – multiple items are naturally a tuple:
args=('Masala', 2)    # Two items – tuple
args=('Masala', 2, 'extra') # Three items – tuple

# Memory trick:
# Single item → always add trailing comma: args=(val,)
# Multiple items → commas between items: args=(a, b, c)
```

Expected Output

```
Masala chai brewing...
Ginger chai brewing... <- Both start together
Masala chai ready!     <- 2 seconds later
Ginger chai ready!     <- 3 seconds later
Both chais ready!
```

■ Section 5 — Example 3: Threading with I/O (07_thread_download.py)

Scenario — Where Threading Shines

We want to download 3 images from a web server. Each download takes ~1.5 seconds because the thread sends the request and then **waits** for the server to respond.

Sequential: $1.5 + 1.5 + 1.5 = \mathbf{4.5 \text{ seconds}}$

With threading: all 3 requests in-flight simultaneously = **~1.5 seconds**

Full Code — 07_thread_download.py

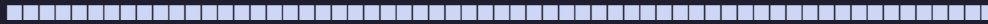
[illegible]

```
for t in threads:
    t.join()

print(f'All downloads done in {time.time()-start:.2f}s')
```

Why Threading Works Here — GIL Released on I/O

WHAT ACTUALLY HAPPENS:



Thread 1 → Sends GET request → WAITING for server response
(GIL RELEASED – no Python code executing)

Thread 2 → Acquires GIL → Sends GET request → WAITING
(GIL RELEASED)

Thread 3 → Acquires GIL → Sends GET request → WAITING
(GIL RELEASED)

All 3 requests are now in-flight at the server simultaneously!

Server responds → Thread 1 receives data → writes to memory A

Server responds → Thread 2 receives data → writes to memory B

Server responds → Thread 3 receives data → writes to memory C

KEY: Each thread writes to its OWN memory location.

NO shared state conflict → GIL is not a bottleneck here!

Result: ~1.5s total instead of ~4.5s sequential

Important caution — shared file names:

If all threads write to the SAME file, you will get a race condition even in I/O operations.

Always ensure each thread writes to a unique destination (different file names, different list indices, different dictionary keys).

■ Section 6 — Where Threading Fails: CPU-Bound Tasks

CPU-Bound Threading Demo

```
import threading, time

def cpu_heavy():
    total = 0
    for i in range(10_000_000):    # 10 million iterations
        total += i                # Pure CPU computation
    print(f'Done: {total}')
```

Two threads – expecting 2x speedup

```
start = time.time()
threads = [threading.Thread(target=cpu_heavy) for _ in range(2)]
for t in threads: t.start()
for t in threads: t.join()
print(f'Threading:      {time.time()-start:.2f}s') # ~1.0s – NO speedup!
```

Compare with sequential

```
start = time.time()
cpu_heavy()
cpu_heavy()
print(f'Sequential:    {time.time()-start:.2f}s') # ~1.0s – same!
```

Why threading gives no speedup for CPU-bound tasks:

During CPU computation, the thread is actively modifying Python objects in memory — it must hold the GIL the entire time. The GIL is only released during I/O waits or after a fixed number of bytecodes (~every 5ms).

With two CPU-bound threads, they constantly compete for the GIL. Only one runs at a time — effectively sequential execution. The added overhead of GIL contention can actually make it *slower* than sequential code.

Solution for CPU-bound tasks: Use multiprocessing. Each process gets its own GIL and runs on a separate CPU core — true parallelism.

Task Type	Example	Best Tool	Reason
I/O Bound	API calls, DB queries	threading	GIL released during wait
I/O Bound	File read/write	threading	GIL released during wait
I/O Bound	Web scraping	threading	GIL released during wait
CPU Bound	Image processing	multiprocessing	Each process owns its GIL

CPU Bound	ML model training	multiprocessing	Each process owns its GIL
CPU Bound	Large calculations	multiprocessing	Each process owns its GIL
Async I/O	Web framework handlers	asyncio	Single-thread event loop

■ Section 7 — Thread Lock & Race Condition (08_thread_lock.py)

The Problem — Without a Lock

```
import threading

counter = 0      # Shared variable – ALL threads see this

def increment():
    global counter
    for _ in range(100_000):
        counter += 1    # UNSAFE – race condition possible!

threads = [threading.Thread(target=increment) for _ in range(10)]
for t in threads: t.start()
for t in threads: t.join()

print(f'Counter: {counter}')    # Expected: 1,000,000
                                # Actual:   ~847,293  WRONG!
```

Why counter += 1 is NOT Atomic

counter += 1 is actually THREE separate steps:

Step 1: READ current value of counter from memory

Step 2: ADD 1 to the read value

Step 3: WRITE the new value back to memory

The GIL can switch between threads between any of these steps. If Thread 1 reads counter=5, then Thread 2 also reads counter=5 (before Thread 1 writes), both compute 6 and both write 6 — one increment is silently lost. This happens millions of times, resulting in a final count far below the expected value.

The Solution — threading.Lock()

```
import threading

counter = 0

lock = threading.Lock()    # Create ONE lock for the counter

def increment():
    global counter
    for _ in range(100_000):
        with lock:          # Acquire lock before modifying
            counter += 1    # SAFE – only one thread at a time
```

```

# Lock auto-released when block exits

threads = [threading.Thread(target=increment) for _ in range(10)]
for t in threads: t.start()
for t in threads: t.join()

print(f'Counter: {counter}') # Always: 1,000,000 ■

```

How with lock: Works Internally

```

# 'with lock:' is equivalent to:
lock.acquire()      # Wait until lock is free, then take it
try:
    counter += 1     # Protected operation
finally:
    lock.release()   # ALWAYS release, even if exception occurs

# 'with lock:' handles this automatically – always use it!

# Step-by-step with 2 threads:
# Thread 1 → lock.acquire() → GETS LOCK
# Thread 1 → reads counter=5, adds 1, writes 6
# Thread 1 → lock.release()
# Thread 2 → lock.acquire() → NOW GETS LOCK (was waiting)
# Thread 2 → reads counter=6, adds 1, writes 7
# Thread 2 → lock.release()
# Result: 7 ■ No increment lost!

```

Lock Best Practices

Practice	Why Important
Use 'with lock:' over manual acquire/release	Guarantees release even if exception occurs
Create ONE lock per shared resource	Multiple locks for same resource causes deadlocks
Keep locked section minimal	Long lock periods reduce concurrency
Never hold a lock while doing I/O	Other threads starved during slow I/O wait
Communicate lock requirement to teammates	All writers of shared data must use the same lock

■ Section 8 — Interview Questions & Model Answers

Q1. What is the difference between a Thread and a Process?

A process is an independent program instance with its own private memory space, its own Python interpreter, and its own GIL. Processes are isolated — one cannot directly read another's memory. A thread is a lightweight unit of execution that lives inside a process. All threads in the same process share memory — they can read and write each other's variables directly. Threads are faster to create than processes but require synchronisation (locks) to prevent race conditions.

Q2. What does `threading.Thread(target=func, args=(a, b))` do? What is `args`?

`threading.Thread()` creates a thread object and tells it which function to execute (`target`) and what arguments to pass to that function (`args`). The `args` parameter must be a tuple. Python unpacks the tuple positionally into the function's parameters. A common mistake is writing `args='value'` — this is just a string in parentheses, not a tuple. For a single argument, the correct form is `args=('value',)` with a trailing comma.

Q3. What is `.start()` and `.join()`? What happens if you skip `.join()`?

`.start()` schedules the thread for execution — it returns immediately while the thread runs concurrently in the background. `.join()` blocks the calling (main) thread until the target thread terminates. If you skip `.join()`, the main thread may complete and the program may exit before the background threads finish their work — producing incomplete or no output. Always call `.start()` before `.join()`; calling `.join()` first raises a `RuntimeError`.

Q4. Why does threading improve performance for I/O-bound tasks but not CPU-bound tasks?

For I/O-bound tasks, threads spend most of their time waiting for external responses (server, database, disk). During this wait, the GIL is voluntarily released, allowing other threads to run. Multiple threads can have their I/O requests in-flight simultaneously, dramatically reducing total time. For CPU-bound tasks, threads are actively executing Python bytecode and modifying memory — they must hold the GIL. The GIL serialises execution, making multiple CPU threads no faster (and sometimes slower) than a single thread.

Q5. What is a Race Condition? Give a concrete example.

A race condition occurs when the correctness of a program depends on the timing of thread execution — leading to unpredictable and often incorrect results. Example: `counter += 1` is three operations: read, add, write. Thread 1 reads `counter=5`. Before Thread 1 writes, Thread 2 also reads `counter=5`. Both compute 6 and both write 6. One increment is silently lost. With 10 threads each doing 100,000 increments, the expected result is 1,000,000 but the actual result may be ~847,000 — different every run.

Q6. How does `threading.Lock()` prevent a race condition?

`threading.Lock()` creates a mutex — only the thread that has acquired the lock can execute the protected block. Using 'with lock:' before `counter += 1` means: acquire the lock, perform all three sub-operations (read, add, write) atomically without interruption, then release the lock. Any other thread that reaches 'with lock:' while the lock is held will block until it is released — ensuring no two threads ever modify counter simultaneously.

Q7. When should you NOT use `threading.Lock()`?

You should not use a lock when threads access completely separate memory locations — no lock is needed because there is no shared state. For example, in the download example, each thread writes its response to a different variable — no lock required. Over-locking reduces concurrency: if every operation acquires the same lock, threads end up running sequentially, defeating the purpose of threading. Use locks surgically — only around the minimum code that accesses shared mutable state.

Q8. What is `threading.current_thread()` and when is it useful?

`threading.current_thread()` returns the Thread object representing the thread that is currently executing the call. The `.name` attribute gives a human-readable label. It is useful for debugging and logging — by including the thread name in log messages, you can trace exactly which thread executed which code path. You can assign custom names at creation: `threading.Thread(target=func, name='Worker-1')`, or Python will assign a default name like `Thread-1`, `Thread-2`, etc.

■ Section 9 — Quick Revision Cheatsheet

One-Liner Definitions

```
Thread          = Lightweight execution unit inside a process, shares memory
Process         = Independent program with own memory and GIL
threading.Thread = Creates a thread – does NOT start it
.start()        = Begin execution (returns immediately)
.join()         = Block caller until this thread finishes
args=(val,)     = Single-item tuple – trailing comma REQUIRED
I/O Bound      = Task spends time waiting – threading shines here
CPU Bound      = Task spends time computing – use multiprocessing
Race Condition  = Two threads corrupt shared data due to no synchronisation
threading.Lock() = Mutex – only one thread can hold it at a time
with lock:      = Acquire lock, run block, auto-release (even on exception)
current_thread() = Returns the Thread object currently executing
```

Complete API Quick Reference

Code	What it does
<code>t = threading.Thread(target=f)</code>	Create thread — not started yet
<code>t = threading.Thread(target=f, args=(a,))</code>	Create thread with single arg
<code>t = threading.Thread(target=f, args=(a,b))</code>	Create thread with multiple args
<code>t = threading.Thread(name="Worker")</code>	Create named thread
<code>t.start()</code>	Start the thread
<code>t.join()</code>	Wait for thread to finish
<code>threading.current_thread().name</code>	Name of currently running thread
<code>lock = threading.Lock()</code>	Create a mutex lock
<code>with lock:</code>	Acquire lock, run code, auto-release
<code>lock.acquire()</code>	Manually acquire lock (blocks if held)
<code>lock.release()</code>	Manually release lock

Common Mistakes to Avoid

```
MISTAKE 1: args=('value')  <- Not a tuple! Just parenthesised string
```

```
FIX      : args=('value',) <- Trailing comma makes it a tuple

MISTAKE 2: Calling join() before start()

FIX      : Always start() first, then join()

MISTAKE 3: Not calling join() at all

FIX      : Always join() every thread you start

MISTAKE 4: Using threading for CPU-heavy computation

FIX      : Use multiprocessing – threading won't speed up CPU tasks

MISTAKE 5: Modifying shared data without a lock

FIX      : Wrap all writes to shared variables with 'with lock:'

MISTAKE 6: Creating a new Lock() inside the function

FIX      : Create ONE lock at module level, share it across threads
```

Two Golden Rules for Threading Interviews:

1. "If your task is **WAITING** — use threading. The GIL is released during I/O, so threads run concurrently."
2. "If your task is **COMPUTING** — use multiprocessing. Threading cannot bypass the GIL for CPU-bound work."